

CatCrypt: From Rust to Cryptographic Security in Lean

Bas Spitters

March 26, 2026

Abstract

We describe the methodology and scope of CatCrypt, a library for machine-checked cryptographic security proofs in Lean. CatCrypt provides an end-to-end pipeline from Rust reference implementations to security proofs in the computational model in Lean. The translation from Rust to Lean is done using the Hax tool.

CatCrypt covers 172 cryptographic protocols and constructions with machine-checked security theorems in the computational model. Of these, 110 have the full Rust-to-Lean pipeline. All bounds have been systematically cross-referenced against their published sources (IETF RFCs, NIST standards, and academic papers). Some proofs were ported from SSProve (Rocq), EasyCrypt, ProVerif, CryptoVerif and Squirrel; most are independent formalisations with no prior machine-checked treatment. CatCrypt also includes a verified Lean implementation of a substantial part of the hax transpiler pipeline.

This work is an experiment of what can be done by a researcher working with GenAI. Until recently, the formalization of *one* protocol required months of expert effort. In contrast, the whole of CatCrypt was developed in a period of two months. Because it was developed with AI, we develop a new methodology to increase confidence that the specifications are correct. Moreover, we will continue to audit the code in the coming months to gain even more confidence in the *specification* of the results.

We hope this work will facilitate the adoption of formal methods in the development of security-critical software. This is especially urgent due to AI’s increased hacking capabilities, the explosion of AI generated software and the ongoing post-quantum transition, which requires the development of new cryptographic protocols and their secure implementation.

1 Introduction

The high-assurance cryptography (HACS) community has developed an impressive array of tools for machine-checked security proofs over the past two decades [Bar+21]. Here, we will focus on the computational model of cryptography, which uses so-called game-hopping [BR06]. CertiCrypt [BGZ09] pioneered probabilistic relational Hoare logic (pRHL) for game-hopping proofs; EasyCrypt [Bar+11; Bar+14] used a similar design with better automation in a standalone tool. CryptHOL [BLS20] develops a similar model in Isabelle/HOL. SSProve [Has+23; LS25b; LS25a] interprets State-Separating Proofs (SSP) [Brz+18] in pRHL in Rocq, providing more modular game-based proofs by separating oracle state from protocol logic.

On the implementation side, the hax tool [Bha+24] extracts Lean [MU21] from Rust reference implementations, and the multi-tool methodology [Bha+25] demonstrates end-to-end verification of cryptographic protocol implementations, combining hax extraction, SSProve-style game-based proofs, symbolic verification in ProVerif and program verification with F*.

This paper describes CatCrypt, a library that brings these threads together: SSP-style game-based proofs and Rust-to-Lean extraction, all in a single framework formalised in Lean. CatCrypt started as a port of Rocq’s SSProve [Has+23], but is entirely standalone now.

We focus on the methodology. Our contributions include:

1. Formalization to 172 cryptographic constructions (most with no prior machine-checked treatment in any tool), including real-world protocols with concrete security bounds (Section 4). A complete list can be found in Section 4. Worked examples appear in the appendix.
2. A methodology to address the specification problem, consisting of: an end-to-end pipeline from Rust reference implementations to security theorems via hax extraction (Section 2), and a formally verified web of implications between security definitions, cross-validated against six standard textbooks, providing independent assurance of specification correctness (Section 2.1).
3. We provide evidence that high-assurance cryptography can be done in one foundational proof assistant (Lean), instead of depending on standalone tools.

2 The Methodology Pipeline

CatCrypt follows the methodology of Bhargavan et al. [Bha+25]: cryptographic protocol implementations are written in Rust, extracted to Lean via hax [Bha+24], and verified against abstract security specifications in the SSProve framework. The pipeline has the following layers:

1. **Standards.** We collect IETF/NIST standards, reference implementations, KAT test vectors and scientific articles with cryptographic proofs.
2. **Rust reference implementation.** These are written in safe, hax compatible, Rust. The Hax library already contains a collection of such reference implementations. Our reference implementations are validated against RFC/NIST test vectors and differential testing against official reference implementations. We use Kani [Van+22] for bounded model checking against official implementations.
3. **Hax extraction.** The hax compiler extracts Rust types and functions into Lean, preserving the computational structure. We have ported a substantial part of the Hax transpiler to Lean and proved it correct (§3.2).
4. **Abstract types and dependencies.** A `Dependencies` typeclass abstracts cryptographic primitives from their implementations. This decouples the security proof from the concrete implementation: the same proof works for any instantiation of the dependencies. This is in line with the usual cryptographic model where one assumes that a primitive has a certain property, like being a PRF. The hax-extracted code is then shown to be an instance of these typeclasses: for example, the extracted HMAC-SHA256 implementation provides a concrete `HKDFDependencies` instance, and its PRF assumption supplies the $\varepsilon_{\text{hkdf}}$ term in the protocol’s security bound.
5. **Game hopping and security theorem.** The security proof is then carried out on the lean translation of the rust reference implementation.
6. **Concrete primitives via libcrux.** For performance, optionally a bridge layer instantiates the abstract assumptions with optimized primitives from the libcrux verified library [KBF23]: X25519 and P-256 for DDH, HKDF-SHA256 for PRF, AES-128 for PRP, and AES-128-GCM for AEAD. Concrete protocol theorems (HPKE, EDHOC, SPQR) combine these bridges with the abstract security proofs. We have generated Hax valid reference implementations of these libcrux primitives. Validation proceeds in the three ways described in 2.

Trusted computing base. The Trusted Computing Base (TCB) consists of four components:

- (i) *Lean kernel.* All proofs are machine-checked by the Lean type checker. The kernel and its dependent type theory are trusted.
- (ii) *Hax extraction* (layer 1→2). The hax compiler translates Rust into Lean. This translation is not itself fully verified; a bug in hax could produce Lean code that does not faithfully represent the Rust source. However, we have a verified implementation for a large part of

the hax pipeline in Lean (AST, denotational semantics, a number of compilation phases with feature-erasure proofs, and translation to `RawCode`), which reduces the trusted surface of the extraction. Moreover, the generated Lean code is again validated against the test vectors, now written directly in Lean. Moreover, it can be inspected by hand (possibly with the help of AI).

- (iii) *Cryptographic assumptions*. The usual cryptographic hardness assumptions (DDH, PRF, IND-CPA, etc.).
- (iv) *Primitive correctness* (layer 6). The `libcrux` library consists of formally verified code (from `F*` and `Vale`). What must be trusted is that we have correctly translated its specifications from another proof assistant to Lean. This is partially mitigated by test-vector validation (`native_decide`) and differential testing against the original implementations.

2.1 Specification confidence

Lean checks that our proofs are complete, but how do we know we proved the correct theorems? The protocol specifications are grounded in IETF/NIST standards and derived from reference implementations validated against official test vectors, which gives confidence in their correctness. The security proofs are game-based, and many of the games and bounds are published in the literature. We have systematically cross-referenced all 138 security bounds against their published sources (see [Section 6](#)).

To further build confidence in our security definitions, we have constructed a web of formally proved implications between alternative formulations of the same security notion. For instance, we prove that IND-CPA, IND\$ (real-or-random), and semantic security are all equivalent (up to small constant factors); that SUF-CMA implies EUF-CMA (and coincides with it for deterministic schemes); that collision resistance implies both target collision resistance and preimage resistance [RS04]; and that IND-CPA combined with INT-CTXT implies IND-CCA2 (the Bellare–Namprempre theorem [BN00]). Each definition has been carefully cross-validated (with AI) against six textbooks: Boneh–Shoup [BS20], Katz–Lindell [KL20], Rosulek [Ros21], Shoup’s game-playing framework [Sho04], Goldreich’s *Foundations of Cryptography* [Gol01; Gol04], and Cramer–Damgård–Nielsen [CDN15], as well as the EasyCrypt standard library [Bar+14] and the SSProve Rocq formalization [Has+23]. In particular, over 40 examples have been ported from *The Joy of Cryptography* [Ros21], covering 12 of its 16 chapters, including PRF→PRG, PRF→MAC, CBC mode, Encrypt-then-MAC, Merkle–Damgård, universal hashing, and Shamir secret sharing. A bug in any single definition would break at least one implication in the web, providing an additional layer of assurance beyond what any individual cross-check could offer.

3 Framework Architecture

3.1 Computational Cryptography

We follow the usual monadic encoding of computational cryptography. Variants of which can be found in EasyCrypt [Bar+14], CryptHOL’s `spmf` [BLS20], and SSProve [Has+23].

The base computation type is the *stateful sub-probabilistic monad*:

$$\text{SPComp } \alpha := \text{Heap} \rightarrow \text{SDistr } (\alpha \times \text{Heap})$$

where $\text{SDistr } \alpha := \text{PMF}(\text{Option } \alpha)$ is the type of sub-probability distributions (probability mass functions on `Option` α). The heap provides mutable state for oracle implementations; sub-distributions model both probabilistic sampling and failure. In this way, we model an imperative probabilistic language in Lean.

Game-based security is then expressed via *statistical distance*:

$$\text{sdist}(f, g) := \sup_D |\Pr[D \circ f = 1] - \Pr[D \circ g = 1]|$$

where D ranges over all distinguishers. Like in SSProve, we define packages. SSProve defines a deeply embedded domain-specific language for protocols, and a Dijkstra monad-based program logic. In contrast, we work directly with denotational semantics. The SSP methodology is implemented as a reflexive tactic in Lean.

3.2 Verified hax pipeline

The hax transpiler [Bha+24] extracts Rust code into Lean, but it is itself an unverified OCaml program: a bug in hax could silently produce Lean code that does not faithfully represent the Rust source. To reduce this trusted surface, we have ported a substantial part of the hax transpiler pipeline to Lean and proved it correct.

We start our formalisation with an imperative expression AST (`ImpExpr`), which we branch off from the hax frontend. `ImpExpr` models safe Rust: references, mutable assignments, loops, early returns, and the question-mark operator. A fuel-bounded big-step denotational semantics (`denote`) interprets these expressions in a state monad tracking four outcome types (normal value, early return, break, continue), faithfully modelling Rust’s non-local control flow. Four feature-erasure compilation phases are then verified: (1) *dropReferences* eliminates borrows and derefs, exploiting the absence of aliased mutable references in safe Rust; (2) *localMutation* rewrites mutable assignments into functional let-rebinding; (3) *functionalizeLoops* converts `for/while` loops with `break/continue` into fold combinators; (4) *cfIntoMonads* translates early returns and question-mark into `ControlFlow` patterns. Each phase is proved to preserve the denotational semantics and to establish an inductive feature-absence predicate; the composite `pipeline_correct` theorem guarantees that the fully transformed program denotes identically to the original.

After the four phases, a translation pass converts the resulting *fully functional* expressions into SSProve’s typed `RawCode` deep embedding. A key theorem establishes that the output contains no oracle calls, ensuring compatibility with the game-based proof infrastructure. This gives a verified path from the Rust-level AST to the SSProve computation monad, reducing the hax TCB to the front-end parser and the phases not yet ported.

4 Protocol Portfolio

CatCrypt covers 172 cryptographic protocols and constructions with machine-checked security theorems, all backed by game-hopping reductions. Of these, 110 have the full Rust-to-Lean pipeline (hax-compatible Rust crate, extraction, type bridge, security theorem, and Rust and Lean test validation). The remaining ones have grounded security bounds matching the literature, but lack a dedicated hax crate because we were unable to find a reference implementation in Rust or public test vectors.

Of these, 132 have no prior treatment in a general-purpose proof assistant (EasyCrypt, CryptHOL, SSProve, FCF, or IPDL); 15 have prior analyses only in specialized tools (CryptoVerif, Tamarin, or DY*). All our proofs are in the computational model. Notable coverage milestones include: all 14 NIST PQC Additional Signatures Round 2 candidates, the complete eIDAS 2.0 identity stack (BBS, SD-JWT, OpenID4VP, mDL), all three ISO/IEC 28033 FHE families (BGV, TFHE, CKKS), four next-generation ZK proving systems (Jolt, Circle STARK, Plonky3, Binius), and three major E2E messaging platforms (Signal, Matrix/Vodozamac, iMessage PQ3).

Prior formalization abbreviations: EC = EasyCrypt [Bar+14], CH = CryptHOL [BLS20], SR = SSProve/Rocq [Has+23], PV = ProVerif [Bla01], CV = CryptoVerif [Bla06], Tam = Tamarin [Mei+13], DY = DY* [Bha+21].

Key exchange & transport (30 protocols).

- **EDHOC** (RFC 9528). Lightweight authenticated key exchange for constrained IoT devices ($3\varepsilon_{\text{DDH}} + 4\varepsilon_{\text{HKDF}} + 2\varepsilon_{\text{AEAD}}$) (cf. RFC 9528 [SMP24]). Prior: PV [Bru+18; Jac+23], Tam [Jac+23].
- **TLS 1.3** (RFC 8446). Transport layer security: full 5-stage key schedule composition, transcript hash chain under the random oracle model, record-layer AEAD, and Finished MAC. Prior: PV [BBK17], CV [BBK17], Tam [Cre+17], DY [Bha+21], SR [Bha+25].
- **HPKE** (RFC 9180). Hybrid public-key encryption with base, PSK, Auth, and AuthPSK modes. Includes labeled KDF glue (RFC 9180 §4), `kem_context` composition, multi-message security, and secret export ($\varepsilon_{\text{DDH}} + 2\varepsilon_{\text{HKDF}} + \varepsilon_{\text{AEAD}}$) ([Alw+21], Theorem 1). Prior: CV [Alw+21].
- **HPKE Auth** (RFC 9180). Authenticated HPKE mode with insider security. Prior: CV [Alw+21].
- **MLS** (RFC 9420). Messaging Layer Security: TreeKEM key agreement ($\text{pathLen} \times \varepsilon_{\text{HPKE}}$), key schedule ($12\varepsilon_{\text{HKDF}}$) ([ARJ20; BBN22]), all 7 proposal types (Add, Remove, Update, PSK, ReInit, ExternalInit, GroupContextExtensions), two-layer application message framing ($Q(\varepsilon_{\text{ct}} + \varepsilon_{\text{sd}})$), and symbolic TreeSync commit authentication. Novel.
- **SSH** (RFC 4253). Secure shell transport with DH key exchange and AEAD ($\varepsilon_{\text{DDH}} + \varepsilon_{\text{HKDF}} + \varepsilon_{\text{AEAD}} + \varepsilon_{\text{MAC}}$) ([CB13]). Prior: CV [CB13].
- **WireGuard**. Modern VPN protocol with Noise-based handshake ($4\varepsilon_{\text{GapDH}} + \varepsilon_{\text{HKDF}} + 3\varepsilon_{\text{AEAD}}$) ([LBB19]). Prior: CV [LBB19], Tam [Bas+25].
- **Noise IK/XX/KK/NK** (4 patterns). Noise handshake patterns with varying authentication levels: NK ($2\varepsilon_{\text{GapDH}} + \varepsilon_{\text{HKDF}} + \varepsilon_{\text{AEAD}}$), KK ($2\varepsilon_{\text{GapDH}} + \varepsilon_{\text{HKDF}} + \varepsilon_{\text{AEAD}}$), IK ($4\varepsilon_{\text{GapDH}} + \varepsilon_{\text{HKDF}} + \varepsilon_{\text{AEAD}}$), XX ($3\varepsilon_{\text{GapDH}} + \varepsilon_{\text{HKDF}} + \varepsilon_{\text{AEAD}}$). Prior: PV [KNB19].
- **Kerberos V** (RFC 4120). Ticket-based network authentication protocol. Prior: CV [Bla+08].
- **OEKE**. Online encrypted key exchange (password-authenticated). Prior: CV [Bla12].
- **OPAQUE** (RFC 9807). Asymmetric PAKE with OPRF-based credential storage ($\varepsilon_{\text{OPRF}} + \varepsilon_{\text{HKDF}} + \varepsilon_{\text{3DH}} + q_s/|\mathcal{D}|$; aPAKE bound with online dictionary term). Novel.
- **SPAKE2** (RFC 9382). Symmetric password-authenticated key exchange. Novel.
- **SRP** (RFC 5054). Secure remote password protocol. Novel.
- **Signal X3DH**. Extended triple Diffie-Hellman key agreement. Prior: DY [Bha+21].
- **Signal SPQR**. Post-quantum ratcheting with KEM-based key evolution. Novel.
- **ACME** (RFC 8555). Automated certificate management: domain validation soundness ($\varepsilon_{\text{EUFCMA}} + \varepsilon_{\text{TLS}}$). Novel.
- **Signal Double Ratchet**. Continuous key agreement with forward-secure message encryption. The symmetric ratchet chain has bound $n_{\text{epochs}} \cdot \varepsilon_{\text{PRG}}$; composing with the DH ratchet (CKA) gives $n_{\text{epochs}} \cdot (\varepsilon_{\text{DDH}} + \varepsilon_{\text{PRG}})$ ([ACD19]). Novel.
- **CPace** (IETF draft). Single-round password-authenticated key exchange (ε_{DDH}). Novel.
- **FIDO2** (WebAuthn). Challenge-response device authentication with public-key credentials (ε_{sig}). Prior: EC [Che+21], Tam [Gua+22].
- **QUIC** (RFC 9001). UDP-based transport: packet protection IND-CPA from AEAD ($\varepsilon_{\text{AEAD}}$). Novel.
- **DTLS 1.3** (RFC 9147). Datagram TLS: epoch-based key schedule + per-packet AEAD ($\varepsilon_{\text{KE}} + n_{\text{epochs}} \cdot \varepsilon_{\text{AEAD}}$). Novel.
- **IPsec ESP** (RFC 4303). IP-layer encryption: IKE key exchange + per-packet AEAD ($\varepsilon_{\text{IKE}} + n_{\text{pkts}} \cdot \varepsilon_{\text{AEAD}}$). Novel.
- **ECH** (draft-ietf-tls-esni). Encrypted Client Hello: HPKE-encrypted SNI for TLS 1.3 privacy ($\varepsilon_{\text{HPKE}} + \varepsilon_{\text{TLS}}$). Prior: PV [BCW22].
- **Hybrid TLS ML-KEM** (draft-ietf-tls-ecdhe-mlkem). Post-quantum hybrid key exchange combining X25519 with ML-KEM-768 ($\varepsilon_{\text{classical}} + \varepsilon_{\text{PQ}} + \varepsilon_{\text{KDF}}$). Novel.
- **Hybrid SSH KEM** (draft-ietf-sshm-mlkem-hybrid-kex). Post-quantum hybrid key ex-

change for SSH ($\varepsilon_{\text{classical}} + \varepsilon_{\text{PQ}} + \varepsilon_{\text{hash}}$). Novel.

- **OHTTP** (RFC 9458). Oblivious HTTP: HPKE-based relay privacy for HTTP requests ($\varepsilon_{\text{HPKE}}$). Prior: Tam [Clo22].
- **KEMTLS**. KEM-authenticated TLS 1.3 without handshake signatures ($\varepsilon_{\text{KEM,eph}} + \varepsilon_{\text{KEM,static}} + \varepsilon_{\text{HKDF}}$) ([SSW20]). Novel.

Signatures & authentication (12 protocols).

- **Schnorr**. Discrete-log signature with forking lemma. Prior: EC [Bar+11], CH [BLS20].
- **EdDSA** (RFC 8032). Edwards-curve digital signature algorithm. Prior: EC [Bar+11] (partial).
- **ECDSA** (FIPS 186). Elliptic curve digital signature algorithm. Novel.
- **BLS/BBS** (IETF draft). Pairing-based signatures with selective disclosure. Novel.
- **EUDIW/BBS** (eIDAS 2.0). BBS-based verifiable credentials for EU digital wallet. Novel.
- **MuSig2** (BIP 327). Two-round Schnorr multi-signature scheme. Novel.
- **Chaum blind signatures**. Blind signatures for anonymous tokens. Novel.
- **Ring signatures**. Signer-anonymous signatures within an ad-hoc group. Novel.
- **Group signatures**. Anonymous group signatures with traceability. Novel.
- **FDH**. Full-Domain Hash signature from trapdoor permutations. Prior: EC [Bar+14], CH [BLS20], CV [Bla06].
- **RSA-PSS** (PKCS#1 v2.2). Probabilistic RSA signature with salt. Novel.
- **SD-JWT VC** (RFC 9901). Selective disclosure JWT verifiable credentials for eIDAS 2.0 ($\varepsilon_{\text{sig}} + n \cdot \varepsilon_{\text{hash}}$). Novel.

Post-quantum cryptography (21 protocols).

- **ML-KEM** (FIPS 203). Module-lattice key encapsulation: IND-CCA via FO transform, binding properties (LEAK-BIND-K-PK, LEAK-BIND-CT-K) [CDM24], anonymity (IK-CCA), and formal decryption failure bounds. Prior: EC [Alm+24].
- **ML-DSA** (FIPS 204). Module-lattice digital signature. Prior: EC [Bar+14] (Dilithium).
- **SLH-DSA** (FIPS 205). Stateless hash-based digital signature. Prior: EC [Bar+24].
- **FN-DSA** (FIPS 206). Fast-Fourier lattice-based digital signature (Falcon). Novel.
- **FAEST**. Signature via VOLE-in-the-Head from symmetric primitives. Novel.
- **Picnic**. Zero-knowledge signature from MPC-in-the-head. Novel.
- **HAWK**. Lattice-based signature. Novel.
- **UOV**. Unbalanced Oil and Vinegar multivariate signature. Novel.
- **MAYO**. Multivariate signature with short keys. Novel.
- **SQIsign**. Signature from supersingular isogenies. Novel.
- **XMSS** (RFC 8391). Extended Merkle Signature Scheme, stateful hash-based. Prior: EC [Bar+23].
- **HQC** (NIST round 4). Hamming Quasi-Cyclic code-based KEM. Novel.
- **Hybrid signatures**. Dual classical and post-quantum signing for transition period. Novel.
- **CROSS**. Code-based signature from restricted syndrome decoding + MPCitH. Novel.
- **SDitH**. Syndrome Decoding in the Head signature. Novel.
- **LESS**. Linear Equivalence Signature Scheme from code equivalence ($\varepsilon_{\text{LCEP}} + \varepsilon_{\text{ROM}}$). Novel.
- **SNOVA**. Structured Non-commutative Oil-Vinegar signature over $\text{GF}(16)$ matrix rings ($\varepsilon_{\text{NCMQ}} + \varepsilon_{\text{ROM}}$). Novel.
- **Mirath**. MinRank-based signature via MPCitH ($\varepsilon_{\text{MinRank}} + \varepsilon_{\text{MPCitH}} + \varepsilon_{\text{ROM}}$). Novel.
- **PERK**. Permutation kernel signature ($\varepsilon_{\text{PKP}} + \varepsilon_{\text{ROM}}$). Novel.
- **RYDE**. Rank Syndrome Decoding signature over $\mathbb{F}_{q^m}$ ($\varepsilon_{\text{RSD}} + \varepsilon_{\text{MPCitH}} + \varepsilon_{\text{ROM}}$). Novel.

- **MQOM.** MQ-based signature via MPC-in-the-Head ($\varepsilon_{\text{MQ}} + \varepsilon_{\text{MPCitH}} + \varepsilon_{\text{ROM}}$). Novel.

Together with the 6 already listed (FAEST, HAWK, UOV, MAYO, SQIsign, Picnic), this gives **all 14 NIST PQC Additional Signatures Round 2 candidates** plus CROSS, SDitH, and hybrid signatures.

Encryption, AEAD & KEM (20 protocols).

- **ElGamal.** IND-CPA public-key encryption under DDH. Prior: EC [Bar+14], CH [BLS20], SR [Has+23].
- **Hashed ElGamal.** IND-CPA in the ROM. Prior: EC [Bar+14], CH [BLS20].
- **Cramer-Shoup.** IND-CCA2 from DDH + target collision resistance. Prior: EC [Bar+14].
- **KEM-DEM.** Generic KEM+DEM composition for IND-CCA PKE. Prior: EC [Bar+14].
- **Saber.** Lattice-based KEM from Module-LWR. Prior: EC [HMS22].
- **X-Wing.** Post-quantum hybrid KEM combining X25519 with ML-KEM-768. Novel.
- **AES-GCM** (SP 800-38D). AES in Galois/Counter Mode authenticated encryption. Novel.
- **AES-CCM** (SP 800-38C). Counter with CBC-MAC authenticated encryption. Novel.
- **AES-SIV** (RFC 5297). Misuse-resistant AES-based deterministic AEAD. Novel.
- **AES-GCM-SIV** (RFC 8452). Nonce-misuse-resistant AEAD. Novel.
- **AEGIS** (IRTF draft). AES-round-based high-speed AEAD. Novel.
- **Ascon** (NIST LWC). Lightweight AEAD, NIST standard for constrained devices. Novel.
- **ChaCha20-Poly1305** (RFC 8439). Stream-cipher AEAD for TLS and VPN. Novel.
- **ChaCha20** (RFC 8439). ChaCha20 block function as PRF. Novel.
- **Proxy re-encryption.** ElGamal-based proxy re-encryption (BBS98). Novel.
- **Boneh-Franklin IBE.** Identity-based encryption from bilinear pairings. Novel.
- **CP-ABE.** Ciphertext-policy attribute-based encryption from DBDH. Novel.
- **BGV FHE.** Fully homomorphic encryption from Ring-LWE. Novel.
- **TFHE.** Torus fully homomorphic encryption with programmable bootstrapping ($\varepsilon_{\text{LWE}} + n \cdot \varepsilon_{\text{bootstrap}}$) ([Chi+20]). Novel.
- **CKKS.** Approximate FHE for arithmetic on encrypted real numbers (modulusChain $\cdot \varepsilon_{\text{RLWE}}$) ([Che+17]). Novel.

Together with BGV, this covers **all three ISO/IEC 28033 FHE families**.

Multi-party computation & secret sharing (15 protocols).

- **Maurer MPC.** Information-theoretic MPC over $\mathbb{Z}/p$, passive ($t < n/2$) and active ($t < n/3$). Prior: EC [Haa+18].
- **SPDZ.** Actively secure MPC with authenticated secret shares. Novel.
- **MASCOT.** OT-based preprocessing for SPDZ MPC. Novel.
- **MAESTRO.** Three-party replicated-sharing MPC for AES evaluation. Novel.
- **FROST** (RFC 9591). Flexible round-optimised threshold Schnorr signatures. Novel.
- **Shamir secret sharing.** Threshold secret sharing over finite fields. Prior: EC [Bar+14] (partial).
- **Feldman VSS.** Verifiable secret sharing with discrete-log commitments. Novel.
- **Threshold decryption.** Threshold ElGamal with Shamir-based key sharing. Novel.
- **Simplest OT.** DDH-based 1-out-of-2 oblivious transfer. Novel.
- **IKNP OT extension.** Amortised oblivious transfer from base OT and PRG. Novel.
- **PSI.** Private set intersection via Diffie-Hellman. Novel.
- **SoftSpoken OT.** Silent VOLE-based oblivious transfer with sublinear communication. Novel.
- **PPSS.** Password-protected secret sharing with threshold recovery. Novel.
- **Threshold ECDSA.** Threshold ECDSA signing with Shamir-based key sharing. Novel.

- **Threshold ML-DSA.** Threshold signing for ML-DSA (NIST IR 8214C) ($\varepsilon_{\text{ML-DSA}} + \varepsilon_{\text{DKG}}$). Novel.

Zero-knowledge & commitments (17 protocols).

- **Bulletproofs.** Logarithmic-size range proofs from discrete log. Novel.
- **Groth16.** Pairing-based zkSNARK with constant-size proofs. Novel.
- **PLONK.** Universal zkSNARK using polynomial commitments. Novel.
- **FRI.** Fast Reed–Solomon interactive oracle proof of proximity. Novel.
- **KZG.** Kate–Zaverucha–Goldberg polynomial commitment: correctness, polynomial binding, evaluation binding, knowledge soundness ([KZG10]). Novel.
- **Pedersen commitment.** Perfectly hiding commitment with CRS trapdoor. Prior: EC [Bar+14].
- **Sigma protocols.** Generic completeness, SHVZK, special soundness + Chaum–Pedersen DLEQ. Prior: EC [Bar+14], CH [BLS20].
- **Fiat-Shamir transform.** Generic sigma-to-signature with q -query forking lemma ([BN06]). Prior: EC [Bar+14].
- **Sigma $\rightarrow$ commitment.** Generic sigma protocol to commitment transform. Prior: EC [Bar+14].
- **Halo2.** Recursive proof composition without trusted setup. Novel.
- **Nova.** Folding-based incrementally verifiable computation. Novel.
- **Marlin.** Universal preprocessing zkSNARK with polynomial commitments. Novel.
- **STARK.** Scalable transparent argument of knowledge from FRI. Novel.
- **Jolt.** zkVM via Lasso lookup argument and sumcheck ($\varepsilon_{\text{sumcheck}} + \varepsilon_{\text{commitment}}$) ([AST24]). Novel.
- **Circle STARK (Stwo).** STARK prover over Mersenne-31 circle groups ($\varepsilon_{\text{FRI-SZ}} + \varepsilon_{\text{hash}}$) ([HLP24]). Novel.
- **Plonky3.** Modular field/hash-agnostic polynomial IOP toolkit ($\varepsilon_{\text{PIOP}} + \varepsilon_{\text{commitment}}$) ([Pol24]). Novel.
- **Binius.** Binary tower field proving system ($\varepsilon_{\text{binary-FRI}} + \varepsilon_{\text{commitment}}$) ([DP23]). Novel.

Messaging (3 protocols).

- **Vodozamac** (Matrix Olm/Megolm). Olm double ratchet + Megolm group ratchet for Matrix E2E encryption ($\varepsilon_{\text{DH}} + \varepsilon_{\text{HKDF}} + n \cdot \varepsilon_{\text{AEAD}}$). Prior: PV [GN24].
- **iMessage PQ3.** Apple Level 3 post-quantum messaging with continuous ML-KEM rekeying ($\varepsilon_{\text{ECDH}} + \varepsilon_{\text{ML-KEM}} + n \cdot \varepsilon_{\text{rekey}} + m \cdot \varepsilon_{\text{AEAD}}$). Prior: Tam [LSB25].
- **OpenID4VP.** Verifiable presentation protocol for eIDAS 2.0 ($\varepsilon_{\text{sig}} + \varepsilon_{\text{nonce}}$). Novel.

Privacy (3 protocols).

- **Piano PIR.** Single-server private information retrieval with sublinear queries ($\varepsilon_{\text{LWE}} + \varepsilon_{\text{hint}}$) ([Zho+24]). Novel.
- **Semaphore.** Zero-knowledge group membership for Ethereum ($\varepsilon_{\text{hash}} + \text{depth} \cdot \varepsilon_{\text{Merkle}}$). Novel.
- **RLN.** Rate-limiting nullifier with Shamir-based slashing ($\varepsilon_{\text{Semaphore}} + \varepsilon_{\text{Shamir}}$). Novel.

Identity & credentials (2 protocols).

- **mDL** (ISO 18013-5). Mobile driving license with issuer COSE signatures and device authentication ($\varepsilon_{\text{issuer}} + \varepsilon_{\text{device}}$). Novel.
- **KEMTLS.** Already listed under key exchange.

Symmetric, hash & MAC (10 protocols).

- **CMAC** (SP 800-38B). CBC-MAC authentication with PRP reduction. Prior: EC [Bar+18].
- **SHA-3 / Sponge** (FIPS 202). Sponge indistinguishability ($\varepsilon_{\text{RP}} + q^2/2^c$). Prior: EC [BB+19].
- **Merkle–Damgård**. Iterated hash collision resistance. Prior: EC [Bar+14].
- **CBC-MAC**. Fixed-length MAC from block cipher. Prior: EC [Bar+14].
- **Universal hashing**. ε -almost-universal hash families (Carter–Wegman). Prior: EC [Bar+14].
- **PRP/PRF switching**. Birthday-bound switching lemma ($q(q-1)/2N$). Prior: EC [Bar+14], CH [BLS20].
- **PRF→PRG**. Counter-mode PRG from PRF. Prior: EC [Bar+14], SR [Has+23].
- **PRF→MAC**. MAC from PRF. Prior: EC [Bar+14], SR [Has+23].
- **PRF→IND-CPA**. IND-CPA encryption from PRF. Prior: EC [Bar+14], SR [Has+23].
- **HMAC** (RFC 2104). Keyed-hash MAC as PRF. Prior: FCF [PM15], VST [Ber+15].

E-voting & verifiable computation (3 protocols).

- **ElectionGuard**. End-to-end verifiable election with homomorphic tallying. Novel.
- **Helios/Belenios**. Ballot privacy ($n \cdot \varepsilon_{\text{IND-CPA}} + n \cdot \varepsilon_{\text{ZK}}$), verifiability, and accountability. Prior: EC [Cor+18].
- **OVN** (Open Vote Network). Blockchain-based boardroom voting with DDH privacy. Prior: SR [Has+23].

Other primitives & constructions (13 protocols).

- **Garbled circuits** (Yao). Half-gate garbling with row reduction and free XOR. Prior: EC [Alm+17b].
- **CryptoBox** (NaCl PKAE). Public-key authenticated encryption via X25519 + XSalsa20-Poly1305. Prior: EC [Bar+14].
- **GPV08 lattice IBE**. Identity-based encryption from LWE. Prior: EC [BB+21].
- **PCR** (Private Count Retrieval). Information-theoretic PIR variant. Prior: EC [SV17].
- **Merkle tree**. Authenticated data structure with collision-resistance security. Novel.
- **Path ORAM**. Access-pattern-hiding oblivious RAM. Novel.
- **SSE**. Searchable symmetric encryption with update privacy. Novel.
- **GBF-PSI**. Private set intersection via garbled Bloom filters and OPRF. Novel.
- **Wesolowski VDF**. Verifiable delay function from groups of unknown order. Novel.
- **VRF** (IETF draft). Verifiable random function with unique provable outputs. Novel.
- **OTP**. One-time pad: perfect secrecy. Prior: SR [Has+23].
- **Encrypt-then-MAC**. IND-CCA from IND-CPA + INT-CTXT (Bellare–Namprempre). Prior: EC [Bar+14], SR [Has+23].
- **Cascade PRF**. Double encryption from PRF. Prior: EC [Bar+14].

5 Related Work

Extraction pipeline. Jasmin [Alm+17a] is a framework for developing high-speed, high-assurance cryptographic software: the Jasmin language gives the programmer assembly-level control, while its Rocq-verified compiler guarantees memory safety and constant-time execution. The security proofs are carried out in EasyCrypt. The Last Yard [Has+24] closes the gap between Jasmin implementations and cryptographic security proofs by connecting them to SSProve (Rocq) via hax, establishing end-to-end verification for high-speed cryptography.

Computational proof tools. In addition to the tools for computational cryptography already mentioned, IPDL [Gan+23] provides equational reasoning for concurrent reactive pro-

ocols, and FCF [PM15] formalises game-based proofs in Rocq with a focus on foundational constructions.

Scope comparison. We have counted distinct protocols and constructions formalised in each framework (from GitHub repositories and the literature):

Framework	Proof assistant	Protocols	Sources
EasyCrypt	standalone	~74	std. lib + Formosa-Crypto + ~30 papers
CryptHOL	Isabelle	~27	4 AFP entries + papers
SSProve	Rocq	~22	examples/ + 3 papers
FCF	Rocq	~8	examples/ + thesis
CatCrypt	Lean	172	this work

CatCrypt covers all EasyCrypt standard-library protocols and most Formosa-Crypto targets. On a small sample, CatCrypt proofs are approximately 70% shorter than EasyCrypt proofs.

Symbolic methods. A complementary line of work focuses on symbolic protocol verification. ProVerif [Bla01] provides fully automatic verification of authentication and secrecy in the Dolev-Yao model. CryptoVerif [Bla06] provides proofs in the computational model with a high degree of automation. Tamarin [Mei+13; Bas+25] supports both unbounded sessions and has been used to find practical vulnerabilities. DY* [Bha+21] provides modular symbolic verification in F*. Squirrel [Bae+21] implements the CCSA (Computationally Complete Symbolic Attacker) paradigm, where computational soundness holds by construction. Connecting the results of these specialised tools with game-based proofs in a general-purpose proof assistant remains an open challenge.

Autoformalization. Large language models have opened a new avenue for translating informal mathematics into machine-checked proofs. Wu et al. [Wu+22] demonstrated that LLMs can translate competition problems into Isabelle, and Jiang et al. [Jia+23] introduced the Draft-Sketch-Prove pipeline, where an LLM drafts an informal proof, translates it into a formal sketch with `sorry` holes, and an automated prover fills the gaps. Concurrently with our work, Urban [Urb26] autoformalized 130k lines of Munkres general topology in Mizar. He does not address the specification problem. Moreover, the goal of his project was to formalize a well-organized mathematical textbook. Whereas, our goal is focused on the verification of cryptographic software and protocols, which requires interactive development.

6 GenAI Methodology

CatCrypt was developed with extensive use of GenAI. All Lean code was written by the AI, with human direction, design and review. The development proceeded in phases. First, we ported the SSProve framework from Rocq to Lean, building on Mathlib’s existing probability theory and algebraic hierarchy. This required significant human guidance to establish the right design, abstractions and automation. Once the core framework was solid, we ported existing formalised proofs from SSProve (Rocq), EasyCrypt, CryptoVerif and Squirrel. These ported proofs provided enough examples for the AI to learn the patterns of CatCrypt, enabling it to autoformalise new protocols directly from IETF/NIST standards and the scientific literature. Still the whole process required a few months of intense interactive development, during which we gained many new insights, some of which will be published separately.

Our pipeline gives high confidence that the implementations of the protocols are correct. The security bounds in our proofs are checked against published bounds in the literature or against

previous formalisations, where available. The specifications may still contain errors. The author and GenAI found a number of specification errors in the early stages of development. Those we found were usually quickly fixed. The size of the library and the reuse of definitions and assumptions across many protocols provides additional confidence that the specifications are correct. GenAI was also used in the writing process.

7 Sources

The CatCrypt sources are in a private GitHub repository, which we plan to make public as soon as we have reorganized it in a way where we can release individual protocols.

8 Conclusion

CatCrypt demonstrates that machine-checked security proofs for real-world cryptographic protocols can be developed at scale. The combination of Rust extraction via hax, game-based security proofs, and domain-specific automation enables an end-to-end pipeline from implementation to security theorem.

CatCrypt builds on decades of work by the HACS community, especially the open source verified cryptographic protocol theory, which was likely used in the LLM training data, and more directly used for porting purposes.

As argued in [PS24], proposed cryptographic (NIST/IETF) standards should be accompanied by machine-checked proofs of security, preferably in the computational model, and reference implementations should be verified against these proofs. The methodology presented here suggests that this can be done at scale.

9 Acknowledgement

This work was partially funded by Ethereum’s zkEVM Formal Verification Project.

References

- [ACD19] Joël Alwen, Sandro Coretti, and Yevgeniy Dodis. “The Double Ratchet: Security Notions, Proofs, and Modularization for the Signal Protocol”. In: *EUROCRYPT*. Vol. 11476. LNCS. 2019, pp. 129–158. DOI: [10.1007/978-3-030-17653-2_5](https://doi.org/10.1007/978-3-030-17653-2_5).
- [Alm+17a] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. “Jasmin: High-Assurance and High-Speed Cryptography”. In: *CCS*. 2017. DOI: [10.1145/3133956.3134078](https://doi.org/10.1145/3133956.3134078).
- [Alm+17b] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, François Dupressoir, et al. “A Fast and Verified Software Stack for Secure Function Evaluation”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017*. ACM, 2017. DOI: [10.1145/3133956.3134017](https://doi.org/10.1145/3133956.3134017).
- [Alm+24] Jose Bacelar Almeida, Santiago Arranz Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Lechenet, Cameron Low, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, and Pierre-Yves Strub. “Formally Verifying Kyber – Episode V: Machine-Checked IND-CCA Security and Correctness of ML-KEM in EasyCrypt”. In: *CRYPTO*. Vol. 14921. LNCS. 2024. DOI: [10.1007/978-3-031-68379-4_12](https://doi.org/10.1007/978-3-031-68379-4_12).

- [Alw+21] Joël Alwen, Bruno Blanchet, Eduard Hauck, Eike Kiltz, Benjamin Lipp, and Doreen Riepel. “Analysing the HPKE Standard”. In: *EUROCRYPT*. Vol. 12696. LNCS. 2021, pp. 87–116. DOI: [10.1007/978-3-030-77870-5_4](https://doi.org/10.1007/978-3-030-77870-5_4).
- [ARJ20] Joël Alwen, Mariana Raykova, and Daniel Jost. “Key Schedule and Continuous Key Agreement from TreeKEM”. In: *CRYPTO*. 2020.
- [AST24] Arasu Arun, Srinath Setty, and Justin Thaler. “Jolt: SNARKs for Virtual Machines via Lookups”. In: *EUROCRYPT*. 2024. DOI: [10.1007/978-3-031-58751-1_1](https://doi.org/10.1007/978-3-031-58751-1_1).
- [Bae+21] David Baelde, Stéphanie Delaune, Charlie Jacomme, Adrien Koutsos, and Solène Moreau. “An Interactive Prover for Protocol Verification in the Computational Model”. In: *IEEE S&P*. 2021. DOI: [10.1109/SP40001.2021.00078](https://doi.org/10.1109/SP40001.2021.00078).
- [Bar+11] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella-Béguelin. “Computer-Aided Security Proofs for the Working Cryptographer”. In: *CRYPTO*. Vol. 6841. LNCS. 2011, pp. 71–90. DOI: [10.1007/978-3-642-22792-9_5](https://doi.org/10.1007/978-3-642-22792-9_5).
- [Bar+14] Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. “EasyCrypt: A Tutorial”. In: *FOSAD*. 2014. DOI: [10.1007/978-3-319-10082-1_6](https://doi.org/10.1007/978-3-319-10082-1_6).
- [Bar+18] Cécile Baritel-Ruet, François Dupressoir, Pierre-Alain Fouque, and Benjamin Grégoire. “Formal Security Proof of CMAC and Its Variants”. In: *31st IEEE Computer Security Foundations Symposium, CSF 2018*. IEEE, 2018, pp. 91–104. DOI: [10.1109/CSF.2018.00014](https://doi.org/10.1109/CSF.2018.00014).
- [Bar+21] Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. “SoK: Computer-Aided Cryptography”. In: *IEEE S&P*. 2021, pp. 777–795. DOI: [10.1109/SP40001.2021.00008](https://doi.org/10.1109/SP40001.2021.00008).
- [Bar+23] Manuel Barbosa, François Dupressoir, Benjamin Grégoire, Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. “Fixing and Mechanizing the Security Proof of Fiat-Shamir with Aborts and Applications”. In: *Advances in Cryptology – CRYPTO 2023*. Springer, 2023.
- [Bar+24] Manuel Barbosa, François Dupressoir, Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. “A Tight Security Proof for SPHINCS+, Formally Verified”. In: *Advances in Cryptology – ASIACRYPT 2024*. Springer, 2024.
- [Bas+25] David Basin, Cas Cremers, Jannik Dreier, and Ralf Sasse. *Modeling and Analyzing Security Protocols with Tamarin*. Information Security and Cryptography. Springer, 2025. DOI: [10.1007/978-3-031-76584-2](https://doi.org/10.1007/978-3-031-76584-2).
- [BB+19] Manuel Barbosa, Gilles Barthe, et al. “Machine-Checked Proofs for Cryptographic Standards: Indifferentiability of Sponge and Secure High-Assurance Implementations of SHA-3”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019*. ACM, 2019. DOI: [10.1145/3319535.3354241](https://doi.org/10.1145/3319535.3354241).
- [BB+21] Manuel Barbosa, Gilles Barthe, et al. “EasyPQC: Verifying Post-Quantum Cryptography”. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS 2021*. ACM, 2021. DOI: [10.1145/3460120.3484567](https://doi.org/10.1145/3460120.3484567).
- [BBK17] Karthikeyan Bhargavan, Bruno Blanchet, and Nadim Kobeissi. “Verified Models and Reference Implementations for the TLS 1.3 Standard Candidate”. In: *IEEE S&P*. 2017, pp. 483–503. DOI: [10.1109/SP.2017.26](https://doi.org/10.1109/SP.2017.26).
- [BBN22] Karthikeyan Bhargavan, Benjamin Beurdouche, and Prasad Naldurg. “Security Analysis of the MLS Key Derivation”. In: *IEEE S&P*. 2022. DOI: [10.1109/SP46214.2022.9833673](https://doi.org/10.1109/SP46214.2022.9833673).

- [BCW22] Karthikeyan Bhargavan, Vincent Cheval, and Christopher Wood. “A Symbolic Analysis of Privacy for TLS 1.3 with Encrypted Client Hello”. In: *CCS*. 2022, pp. 365–379. DOI: [10.1145/3548606.3559360](https://doi.org/10.1145/3548606.3559360).
- [Ber+15] Lennart Beringer, Adam Petcher, Katherine Q. Ye, and Andrew W. Appel. “Verified Correctness and Security of OpenSSL HMAC”. In: *USENIX Security*. 2015, pp. 207–221.
- [BGZ09] Gilles Barthe, Benjamin Grégoire, and Santiago Zanella-Béguelin. “Formal Certification of Code-Based Cryptographic Proofs”. In: *POPL*. 2009, pp. 90–101. DOI: [10.1145/1480881.1480894](https://doi.org/10.1145/1480881.1480894).
- [Bha+21] Karthikeyan Bhargavan, Abhishek Bichhawat, Quoc Huy Do, Pedram Hosseini, Ralf Küsters, Guido Schmid, and Tim Würtele. “DY*: A Modular Symbolic Verification Framework for Executable Cryptographic Protocol Code”. In: *EuroS&P*. 2021. DOI: [10.1109/EuroSP51992.2021.00052](https://doi.org/10.1109/EuroSP51992.2021.00052).
- [Bha+24] Karthikeyan Bhargavan, Maxime Buyse, Lucas Franceschino, Lasse Letager Hansen, Franziskus Kiefer, Jonas Schneider-Bensch, and Bas Spitters. “hax: Verifying Security-Critical Rust Software using Multiple Provers”. In: *VSTTE*. Cryptology ePrint Archive, Report 2025/142. 2024. DOI: [10.1007/978-3-031-86695-1_7](https://doi.org/10.1007/978-3-031-86695-1_7).
- [Bha+25] Karthikeyan Bhargavan, Lasse Letager Hansen, Franziskus Kiefer, Jonas Schneider-Bensch, and Bas Spitters. “Formal Security and Functional Verification of Cryptographic Protocol Implementations in Rust”. In: *CCS*. Cryptology ePrint Archive, Report 2025/980. 2025. DOI: [10.1145/3719027.3765213](https://doi.org/10.1145/3719027.3765213).
- [Bla+08] Bruno Blanchet, Aaron D. Jaggar, Andre Scedrov, and Joe-Kai Tsay. “Computationally Sound Mechanized Proofs for Basic and Public-Key Kerberos”. In: *ASIACCS*. 2008, pp. 87–99. DOI: [10.1145/1368310.1368326](https://doi.org/10.1145/1368310.1368326).
- [Bla01] Bruno Blanchet. “An Efficient Cryptographic Protocol Verifier Based on Prolog Rules”. In: *IEEE CSFW*. 2001. DOI: [10.1109/CSFW.2001.930138](https://doi.org/10.1109/CSFW.2001.930138).
- [Bla06] Bruno Blanchet. “A Computationally Sound Mechanized Prover for Security Protocols”. In: *IEEE S&P*. 2006. DOI: [10.1109/SP.2006.1](https://doi.org/10.1109/SP.2006.1).
- [Bla12] Bruno Blanchet. “Automatically Verified Mechanized Proof of One-Encryption Key Exchange”. In: *CSF*. 2012, pp. 325–339. DOI: [10.1109/CSF.2012.17](https://doi.org/10.1109/CSF.2012.17).
- [BLS20] David Basin, Andreas Lochbihler, and S. Reza Sefidgar. “CryptHOL: Game-Based Proofs in Higher-Order Logic”. In: *Journal of Cryptology* 33.3 (2020), pp. 494–566. DOI: [10.1007/s00145-019-09341-z](https://doi.org/10.1007/s00145-019-09341-z).
- [BN00] Mihir Bellare and Chanathip Namprempre. “Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm”. In: *ASIACRYPT 2000*. Vol. 1976. LNCS. Springer, 2000.
- [BN06] Mihir Bellare and Gregory Neven. “Multi-signatures in the plain public-key model and a general forking lemma”. In: *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS 2006*. ACM, 2006, pp. 390–399. DOI: [10.1145/1180405.1180453](https://doi.org/10.1145/1180405.1180453).
- [BR06] Mihir Bellare and Phillip Rogaway. “The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs”. In: *EUROCRYPT*. 2006. DOI: [10.1007/11761679_29](https://doi.org/10.1007/11761679_29).
- [Bru+18] Alessandro Bruni, Thorvald Sahl Jørgensen, Theis Grønbech Petersen, and Carsten Schürmann. “Formal Verification of Ephemeral Diffie-Hellman Over COSE (ED-HOC)”. In: *SSR*. Vol. 11322. LNCS. 2018, pp. 21–36. DOI: [10.1007/978-3-030-04762-7_2](https://doi.org/10.1007/978-3-030-04762-7_2).

- [Brz+18] Chris Brzuska, Antoine Delignat-Lavaud, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. “State Separating Proofs: A Reduction Methodology for Real World Protocols”. In: *ASIACRYPT*. Vol. 10993. LNCS. 2018, pp. 222–249. DOI: [10.1007/978-3-030-03326-2_8](https://doi.org/10.1007/978-3-030-03326-2_8).
- [BS20] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. Version 0.6, <https://toc.cryptobook.us/>. 2020.
- [CB13] David Cade and Bruno Blanchet. “From Computationally-Proved Protocol Specifications to Implementations and Application to SSH”. In: *POST*. Vol. 7796. LNCS. 2013, pp. 63–82. DOI: [10.1007/978-3-642-36830-1_4](https://doi.org/10.1007/978-3-642-36830-1_4).
- [CDM24] Cas Cremers, Alexander Dax, and Niklas Medinger. “Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis”. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, CCS 2024*. ACM, 2024.
- [CDN15] Ronald Cramer, Ivan Bjerre Damgård, and Jesper Buus Nielsen. *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press, 2015.
- [Che+17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. In: *ASIACRYPT*. 2017. DOI: [10.1007/978-3-319-70694-8_15](https://doi.org/10.1007/978-3-319-70694-8_15).
- [Che+21] Shan Chen, Samuel Jero, Matthew Jagielski, Alexandra Newkirk, and Jonathan Katz. “Formal Analysis of Access Control Mechanism of 5G Core Network”. In: *Advances in Cryptology – CRYPTO 2021*. FIDO2 EasyCrypt formalization. Springer, 2021.
- [Chi+20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. “TFHE: Fast Fully Homomorphic Encryption Over the Torus”. In: *Journal of Cryptology* 33 (2020), pp. 34–91. DOI: [10.1007/s00145-019-09319-x](https://doi.org/10.1007/s00145-019-09319-x).
- [Clo22] Cloudflare. *Tamarin Analysis of Oblivious HTTP*. <https://github.com/cloudflare/ohttp-analysis>. 2022.
- [Cor+18] Véronique Cortier, Constantin Catalin Dragan, François Dupressoir, and Bogdan Warinschi. “Machine-Checked Proofs for Electronic Voting: Privacy and Verifiability for Belenios”. In: *31st IEEE Computer Security Foundations Symposium, CSF 2018*. IEEE, 2018, pp. 63–77. DOI: [10.1109/CSF.2018.00012](https://doi.org/10.1109/CSF.2018.00012).
- [Cre+17] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. “A Comprehensive Symbolic Analysis of TLS 1.3”. In: *CCS*. 2017, pp. 1773–1788. DOI: [10.1145/3133956.3134063](https://doi.org/10.1145/3133956.3134063).
- [DP23] Benjamin E. Diamond and Jim Posen. *Succinct Arguments over Towers of Binary Fields*. Cryptology ePrint Archive, Report 2023/1784. 2023.
- [Gan+23] Joshua Gancher, Kristina Sojakova, Xiong Fan, Elaine Shi, and Greg Morrisett. “A Core Calculus for Equational Proofs of Cryptographic Protocols”. In: *POPL*. 2023. DOI: [10.1145/3571218](https://doi.org/10.1145/3571218).
- [GN24] Jacob Ginesin and Cristina Nita-Rotaru. “The Matrix Reloaded: A Mechanized Formal Analysis of the Matrix Cryptographic Suite”. In: *CoRR* abs/2408.12743 (2024). arXiv: [2408.12743](https://arxiv.org/abs/2408.12743).
- [Gol01] Oded Goldreich. *Foundations of Cryptography: Volume 1 – Basic Tools*. Cambridge University Press, 2001.
- [Gol04] Oded Goldreich. *Foundations of Cryptography: Volume 2 – Basic Applications*. Cambridge University Press, 2004.

- [Gua+22] Jingjing Guan, Hui Li, Haisong Ye, and Ziming Zhao. “A Formal Analysis of the FIDO2 Protocols”. In: *ESORICS*. Vol. 13556. LNCS. 2022. DOI: [10.1007/978-3-031-17143-7_1](https://doi.org/10.1007/978-3-031-17143-7_1).
- [Haa+18] Helene Haagh, Aleksandr Karbyshev, Sabine Oechsner, Bas Spitters, and Pierre-Yves Strub. “Computer-Aided Proofs for Multiparty Computation with Active Security”. In: *CSF*. 2018, pp. 119–131. DOI: [10.1109/CSF.2018.00016](https://doi.org/10.1109/CSF.2018.00016).
- [Has+23] Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Carmine Abate, Nikolaj Sidorenko, Catalin Hritcu, Kenji Maillard, and Bas Spitters. “SSProve: A Foundational Framework for Modular Cryptographic Proofs in Coq”. In: *ACM Transactions on Programming Languages and Systems* 45.3 (2023), pp. 1–61. DOI: [10.1145/3594735](https://doi.org/10.1145/3594735).
- [Has+24] Philipp G. Haselwarter, Benjamin Grégoire, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Céline Music, Bas Spitters, and Pierre-Yves Strub. “The Last Yard: Foundational End-to-End Verification of High-Speed Cryptography”. In: *CPP*. 2024. DOI: [10.1145/3636501.3636953](https://doi.org/10.1145/3636501.3636953).
- [HLP24] Ulrich Haböck, David Levit, and Shahar Papini. *Circle STARKs*. Cryptology ePrint Archive, Report 2024/278. 2024.
- [HMS22] Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. “Formal Verification of Saber’s Public-Key Encryption Scheme in EasyCrypt”. In: *IACR Cryptol. ePrint Arch.* 2022 (2022), p. 351. URL: <https://eprint.iacr.org/2022/351>.
- [Jac+23] Charlie Jacomme, Elise Klein, Steve Kremer, and Maïwenn Racouchot. “A Comprehensive, Formal and Automated Analysis of the EDHOC Protocol”. In: *USENIX Security*. 2023, pp. 5881–5898. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/jacomme>.
- [Jia+23] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. “Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs”. In: *ICLR*. 2023.
- [KBF23] Franziskus Kiefer, Karthikeyan Bhargavan, and Lucas Franceschino. “HACSpec: A Gateway to High-Assurance Cryptography”. In: *CCS*. 2023. DOI: [10.1145/3576915.3623200](https://doi.org/10.1145/3576915.3623200).
- [KL20] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. 3rd. CRC Press, 2020.
- [KNB19] Nadim Kobeissi, Georgio Nicolas, and Karthikeyan Bhargavan. “Noise Explorer: Fully Automated Modeling and Verification for Arbitrary Noise Protocols”. In: *EuroS&P*. 2019, pp. 356–370. DOI: [10.1109/EuroSP.2019.00034](https://doi.org/10.1109/EuroSP.2019.00034).
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *Advances in Cryptology – ASIACRYPT 2010*. Vol. 6477. LNCS. Springer, 2010, pp. 177–194. DOI: [10.1007/978-3-642-17373-8_11](https://doi.org/10.1007/978-3-642-17373-8_11).
- [LBB19] Benjamin Lipp, Bruno Blanchet, and Karthikeyan Bhargavan. “A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol”. In: *EuroS&P*. 2019, pp. 231–246. DOI: [10.1109/EuroSP.2019.00026](https://doi.org/10.1109/EuroSP.2019.00026).
- [LS25a] Markus Krabbe Larsen and Carsten Schürmann. *Mechanizing Hybrid Arguments in Nominal-SSProve*. IACR ePrint 2025/1122. 2025. URL: <https://eprint.iacr.org/2025/1122>.

- [LS25b] Markus Krabbe Larsen and Carsten Schürmann. “Nominal State-Separating Proofs”. In: *CSF*. 2025. DOI: [10.1109/CSF64896.2025.00038](https://doi.org/10.1109/CSF64896.2025.00038).
- [LSB25] Felix Linker, Ralf Sasse, and David Basin. “A Formal Analysis of Apple’s iMessage PQ3 Protocol”. In: *USENIX Security*. 2025.
- [Mei+13] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. “The TAMARIN Prover for the Symbolic Analysis of Security Protocols”. In: *CAV*. 2013. DOI: [10.1007/978-3-642-39799-8_48](https://doi.org/10.1007/978-3-642-39799-8_48).
- [MU21] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *CADE*. Vol. 12699. LNCS. 2021, pp. 625–635. DOI: [10.1007/978-3-030-79876-5_37](https://doi.org/10.1007/978-3-030-79876-5_37).
- [PM15] Adam Petcher and Greg Morrisett. “The Foundational Cryptography Framework”. In: *POST*. 2015, pp. 53–72. DOI: [10.1007/978-3-319-17127-2_4](https://doi.org/10.1007/978-3-319-17127-2_4).
- [Pol24] Polygon Labs. *Plonky3: A Toolkit for Polynomial IOPs*. <https://github.com/Plonky3/Plonky3>. 2024.
- [PS24] Jonathan Protzenko and Bas Spitters. “Modernizing FIPS for Safe Languages and Verified Libraries”. In: *NIST Workshop on Formal Methods within Certification Programs (FMCP)*. 2024. URL: <https://www.nist.gov/system/files/documents/2024/06/11/08-ProtzenkoSpitters.pdf>.
- [Ros21] Mike Rosulek. *The Joy of Cryptography*. <https://joyofcryptography.com/>. 2021.
- [RS04] Phillip Rogaway and Thomas Shrimpton. “Cryptographic Hash-Function Basics: Definitions, Implications, and Separations for Preimage Resistance, Second-Preimage Resistance, and Collision Resistance”. In: *FSE 2004*. Vol. 3017. LNCS. Springer, 2004.
- [Sho04] Victor Shoup. *Sequences of Games: A Tool for Taming Complexity in Security Proofs*. IACR ePrint 2004/332. 2004. URL: <https://eprint.iacr.org/2004/332>.
- [SMP24] Göran Selander, John Preuß Mattsson, and Francesca Palombini. *EDHOC: Ephemeral Diffie-Hellman Over COSE*. RFC 9528. 2024. DOI: [10.17487/RFC9528](https://doi.org/10.17487/RFC9528).
- [SSW20] Peter Schwabe, Douglas Stebila, and Thom Wiggers. “Post-Quantum TLS Without Handshake Signatures”. In: *CCS*. 2020. DOI: [10.1145/3372297.3423350](https://doi.org/10.1145/3372297.3423350).
- [SV17] Alley Stoughton and Mayank Varia. “Mechanizing the Proof of Adaptive, Information-Theoretic Security of Cryptographic Protocols in the Random Oracle Model”. In: *30th IEEE Computer Security Foundations Symposium, CSF 2017*. IEEE, 2017, pp. 83–99. DOI: [10.1109/CSF.2017.36](https://doi.org/10.1109/CSF.2017.36).
- [Urb26] Josef Urban. “130k Lines of Formal Topology in Two Weeks: Simple and Cheap Autoformalization for Everyone?”. In: *CoRR* abs/2601.03298 (2026). arXiv: [2601.03298](https://arxiv.org/abs/2601.03298).
- [Van+22] Alexa VanHattum, Daniel Schwartz-Narbonne, Nathan Chong, and Adrian Sampson. “Verifying Dynamic Trait Objects in Rust”. In: *ICSE-SEIP*. 2022, pp. 321–330. DOI: [10.1145/3510457.3513031](https://doi.org/10.1145/3510457.3513031).
- [Wu+22] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. “Autoformalization with Large Language Models”. In: *NeurIPS*. 2022.
- [Zho+24] Mingxun Zhou, Andrew Park, Elaine Shi, and Wenting Zheng. “Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation”. In: *IEEE S&P*. 2024. DOI: [10.1109/SP54263.2024.00188](https://doi.org/10.1109/SP54263.2024.00188).